

α Sentiment Analysis

Testing Whether Corporate Disclosure Language Predicts Future Stock Returns

1 Research Question

Most stock-market prediction projects begin with prices, technical indicators, or accounting ratios. This project starts from a different idea: public companies are required to explain their own businesses in detail every year. A Form 10-K report is not marketing copy. It is a regulated document in which management describes business risks, operating results, legal exposure, competition, liquidity, market conditions, and future uncertainty. If a company's outlook is quietly deteriorating, some trace of that deterioration may appear in the way the company writes about itself before it becomes obvious in simple financial ratios.

The narrative of this project is therefore simple and testable. First, collect annual 10-K reports from SEC EDGAR. Second, turn the language in those reports into measurable signals: negativity, uncertainty, litigation language, readability, document length, and year-over-year textual change. Third, link each filing to future stock returns after the actual filing date. Finally, test whether these textual signals help predict which stocks later outperform or underperform the market.

The project is intentionally designed around a scientific research question: can textual features extracted from annual 10-K reports predict future stock returns after controlling for simple market and firm-level baselines? The emphasis is on testing this hypothesis carefully rather than assuming that textual signals must be useful. In addition to future returns, the project also examines related financial outcomes such as future volatility, market-adjusted performance, and the difference between short-term and long-term return horizons.

This research question builds on a literature showing that corporate filings can contain measurable information beyond the numerical statements alone. Loughran and McDonald [1] show why finance-specific dictionaries are necessary for 10-K analysis, because many words that appear negative in general-purpose dictionaries have neutral meanings in accounting and finance. Jegadeesh and Wu [2] further show that the tone of 10-K filings can be connected to market reactions when words are weighted by their empirical relation to returns. Cohen, Malloy, and Nguyen [3] provide a closely related rationale for examining year-over-year textual change: if firms alter the language of their disclosures, those changes may reveal information about future operations that investors do not fully process immediately. Following this line of work, the project examines not only future returns, but also volatility, market-adjusted performance, and differences across return horizons, because disclosure language may indicate both expected performance and uncertainty around that performance.

2 Data Sources

The project uses two main data sources. The 10-K filings and filing metadata are downloaded from SEC EDGAR, which provides the annual reports, filing dates, CIK identifiers, accession

numbers, and document URLs. The financial market data are downloaded with `yfinance`, which provides adjusted stock prices and the SPY benchmark prices used to calculate forward returns, excess returns, and realized volatility.

3 EDGAR Download Procedure

The first operational step of the project is to build a reproducible pipeline for downloading annual 10-K filings from SEC EDGAR. EDGAR does not require an account or an API key, but automated scripts must identify themselves with a user-agent string. In practice, this means setting an environment variable in the terminal before running the downloader:

```
export SEC_USER_AGENT="your name your.email@example.com"
```

The command `export` creates a temporary environment variable for the current terminal session. The variable name `SEC_USER_AGENT` is read by the Python script and sent with each request to the SEC. The value should contain a name and email address so that the script follows SEC access guidelines. The command only affects the active terminal window, if the terminal is closed, the variable must be set again.

The downloader is implemented in `Code/edgar_10k_download.py`. Its purpose is to transform a list of stock tickers into locally stored 10-K filings and a metadata table. The script is divided into small functions because the EDGAR pipeline has several distinct tasks: creating the SEC connection, requesting data, mapping tickers to company identifiers, finding annual filings, downloading documents, and saving the result in a form that later analysis code can use.

The function `sec_session()` starts the process. It reads `SEC_USER_AGENT` from the terminal environment and creates a `requests.Session`. A session is useful because the same HTTP settings can be reused for every request. In this project, that means each SEC request is sent with the same user-agent string, so the script behaves like one identifiable research client.

The helper functions `sec_get()`, `sec_get_json()`, and `sec_get_bytes()` handle communication with SEC servers. `sec_get()` is the central request function. It waits between requests, sends the URL, checks the server response, and retries temporary errors such as 429, 500, 502, 503, and 504. This retry logic matters because large EDGAR downloads can be interrupted by temporary server load or rate limits. `sec_get_json()` is used when the SEC response should become structured Python data, while `sec_get_bytes()` is used when the script downloads the raw filing document itself.

The first SEC data source used by the script is the company ticker mapping:

```
https://www.sec.gov/files/company_tickers.json
```

The function `load_ticker_map()` downloads this file and converts it into a `pandas` table. This table connects stock tickers to CIK numbers. A CIK is the permanent company identifier used by the SEC. Because EDGAR's filing-history endpoint is organized by CIK rather than by ticker, this mapping step is necessary before any 10-K documents can be found. The function `cik_for_ticker()` then looks up one ticker in this table and returns the corresponding CIK. After the ticker has been mapped to a CIK, the script requests the company's submission history from:

```
https://data.sec.gov/submissions/CIK{cik}.json
```

This step is handled by `recent_10k_filings()`. The returned JSON file contains recent filings, including the form type, filing date, accession number, report date, and primary document name. The function converts this JSON structure into a `pandas` table and filters it to keep only annual report forms, specifically 10-K and 10-K/A. The command-line argument `--limit`

controls how many of these annual filings are kept for each company.

For each selected filing, `recent_10k_filings()` also constructs the direct SEC archive URL from three pieces: the CIK, the accession number, and the primary document name. The accession number identifies the filing submission, while the primary document name identifies the actual HTML document inside that submission. This URL is passed to `download_filing()`, which writes the filing document to `data/raw/filings/`. If the file already exists locally, `download_filing()` keeps the existing copy instead of downloading it again. This behavior is important for long runs because the script can be restarted without re-downloading filings that are already saved.

For the full research sample, typing every ticker into the terminal is not practical. Instead, the project uses a ticker-universe file. This is a plain text file with one ticker per line. A small example file is stored at:

```
config/tickers_sample.txt
```

The downloader can read this file directly:

```
python Code/edgar_10k_download.py \  
--tickers-file config/tickers_sp500.txt \  
--limit 10 \  
--metadata-output data/interim/filings_metadata_sp500.csv \  
--sleep-seconds 0.75 \  
--max-retries 8
```

The backslash at the end of the first line is a shell line-continuation character. It allows one long command to be written across several lines. The argument `--tickers-file` tells the script to read tickers from the file instead of requiring them directly in the command. This makes the pipeline easier to scale, because the research universe can be changed by editing a text file rather than editing code.

For a larger version of the project, a file such as `config/tickers_sp500.txt` can be created with the S&P 500 ticker universe. Here, the limit argument requests the ten most recent annual reports per company. The metadata-output argument chooses the name of the CSV file that stores the filing metadata. The sleep-seconds argument controls how long the script waits between SEC requests. The max-retries argument controls how often the script retries temporary SEC errors before marking a filing as failed. These two options make the pipeline more robust for larger downloads, where occasional server errors or rate limits are expected. For a universe of roughly 500 companies, this command would produce approximately 5,000 filings before accounting for missing companies, amended filings, ticker changes, and firms with shorter public histories.

The `read_tickers()` function is responsible for reading the research universe. It can combine tickers passed directly through the command line with tickers stored in a file. It also normalizes the symbols to uppercase and removes duplicates. This keeps the rest of the script simple because the main download loop receives one clean list of tickers.

The `main()` function ties the full procedure together. It reads the command-line arguments, creates the SEC session, loads the ticker map, reads the ticker universe, loops through each company, downloads the selected filings, and collects one metadata row per filing. If a company or filing fails, the script does not stop the entire run. Instead, it records the failure in the metadata with `download_status` and `download_error`. This makes the pipeline easier to audit because successful and failed downloads are visible in one table.

The downloader creates two important outputs. The raw SEC filing documents are saved under:

```
data/raw/filings/
```

The filing metadata are saved in a CSV file such as:

data/interim/filings_metadata_sp500.csv

The metadata file records the ticker, CIK, accession number, filing date, report date, form type, SEC URL, and local file path. This table is the bridge between the raw EDGAR documents and the later stages of the pipeline. After the filings are downloaded, the next step is to clean the HTML text, extract textual features, and link each filing date to future stock returns.

4 Return Data and Label Construction

After the 10-K filings have been downloaded, the next step is to create financial outcome variables for each filing. These variables are called return labels because they describe what happened to the stock after the filing date. The script responsible for this step is:

Code/build_return_labels.py

The script reads the filing metadata CSV produced by the EDGAR downloader. Each row in that file represents one 10-K filing and contains the ticker, accession number, filing date, and local filing path. The filing date is the key event date: all return calculations are measured forward from this date, so the analysis does not use information that would only have been available later.

The command used to build the return labels is:

```
python Code/build_return_labels.py \
--filings data/interim/filings_metadata_sp500.csv \
--output data/processed/return_labels.csv \
--benchmark SPY \
--horizons 30 90 180 252
```

The argument `--filings` tells the script where to find the filing metadata. The argument `--output` chooses the file where the calculated return labels will be saved. The argument `--benchmark SPY` tells the script to use SPY as the market benchmark. The argument `--horizons 30 90 180 252` defines the forward trading-day windows over which returns and volatility are calculated.

The script first downloads adjusted daily closing prices using `yfinance`. Adjusted prices are used because they account for stock splits and dividends, which makes returns more comparable over time. Prices are downloaded separately for each stock ticker and once for the benchmark ticker. The benchmark price series is reused across all filings because the same market benchmark is needed for every company.

A filing date does not always fall on a trading day. For example, a filing can occur on a weekend or after the market has closed. To keep the procedure simple and reproducible, the script uses the first available trading day on or after the filing date as the starting price date. If the filing date is denoted by t , the starting price is therefore:

$$P_{t_0}^i,$$

where t_0 is the first trading day on or after the filing date for company i . For a horizon of h trading days, the ending price is:

$$P_{t_0+h}^i.$$

The forward return for company i over horizon h is calculated as:

$$R_h^i = \frac{P_{t_0+h}^i}{P_{t_0}^i} - 1.$$

The same calculation is applied to the benchmark:

$$R_h^m = \frac{P_{t_0+h}^m}{P_{t_0}^m} - 1,$$

where m denotes the market benchmark, in this project SPY. The excess return is then:

$$ER_h^i = R_h^i - R_h^m.$$

This value measures whether the stock outperformed or underperformed the market after the filing. The script also creates a binary outperformance label:

$$Outperform_h^i = \begin{cases} 1, & \text{if } ER_{i,h} > 0, \\ 0, & \text{if } ER_{i,h} \leq 0. \end{cases}$$

In addition to returns, the project calculates realized volatility. Realized volatility measures how much the stock price fluctuated during the forward window after the filing. First, the script computes daily returns within the window:

$$r_d^i = \frac{P_d^i}{P_{d-1}^i} - 1.$$

Then it calculates the standard deviation of these daily returns and annualizes it using the standard 252-trading-day convention:

$$\sigma_h^i = \sqrt{252} \cdot std(r_d^i).$$

This gives a future realized volatility label for each filing and each horizon. The interpretation is different from return prediction. A high realized volatility value does not necessarily mean the stock performed badly, it means the stock experienced more uncertainty or price movement after the 10-K filing. This is useful because the language in risk disclosures may be more strongly related to future uncertainty than to the direction of returns alone.

The final output of this script is:

`data/processed/return_labels.csv`

This file contains one row per filing. Important columns include the forward return columns for the 30, 90, 180, and 252 day horizons, the corresponding benchmark and excess return columns, the binary outperformance columns, and the realized volatility columns. These labels will later be merged with textual features extracted from the 10-K reports, producing the final modeling dataset.

5 Text Feature Extraction

Once the return labels have been created, the next step is to convert the raw 10-K filings into numerical text features. This step is handled by:

`Code/build_text_features.py`

The script reads the same filing metadata file used in the earlier pipeline steps:

`data/interim/filings_metadata_sp500.csv`

Each row in this metadata file contains a local path to a downloaded SEC filing. The text-feature script opens each local filing file, reads the raw HTML, and converts it into plain text. SEC filings are not clean text documents: they contain HTML tags, formatting instructions, tables, scripts, style definitions, and inconsistent spacing. The function `html_to_text()` removes the HTML structure and drops elements such as scripts, styles, and tables. The remaining text is then normalized so that repeated whitespace becomes a single space. The command for this step is:

```
python Code/build_text_features.py \
  --filings data/interim/filings_metadata_sp500.csv \
  --output data/processed/text_features.csv
```

The argument `--filings` tells the script where to find the EDGAR filing metadata, while `--output` specifies where the resulting feature table should be saved. The output of this step is:

```
data/processed/text_features.csv
```

After the filing has been converted to plain text, the script tokenizes it. In this context, tokenization means splitting the document into individual word units. The function `tokenize()` uses a regular expression to keep alphabetic word-like tokens and convert them to lowercase. This gives the script a consistent representation of the document that can be counted and compared across companies and years.

The core feature calculation is performed by `text_features()`. The first group of features measures document size and complexity. These include the total word count, the number of unique words, the type-token ratio, the average word length, the average sentence length, and the logarithm of the word count. These variables are useful because the length and complexity of a filing may itself reflect the amount of information, risk, or business complexity disclosed by a company.

The second group of features measures finance-related language. The script counts how often words from small finance-oriented dictionaries appear in the filing. It then divides those counts by the total number of words in the document, producing word-share variables. The main variables are `negative_share`, `positive_share`, `uncertainty_share`, and `litigious_share`. These variables are interpretable: a higher uncertainty share, for example, means that a larger fraction of the filing is written using words associated with uncertainty.

The script also adds year-over-year change features with `add_change_features()`. For each company, filings are ordered by filing date, and the script calculates how selected textual features changed relative to the previous filing. Examples include changes in negative language, uncertainty language, litigious language, and document length. These change features are important because many annual reports repeat large portions of prior-year text. A sudden increase in risk or uncertainty language may therefore be more informative than the absolute level of that language in a single filing.

The script records a `feature_status` for every filing. If the text is successfully extracted, the status is `ok`. If the local filing path is missing or the document cannot be processed, the status is `failed`. If the extracted document is unusually short, the status is marked as `suspicious`. This makes the feature table auditable: problematic filings can be inspected without stopping the entire pipeline.

The text-feature file is not the final dataset by itself. It only describes the language of each filing. The final modeling dataset is created after these textual features are merged with the return labels. That merge is done by:

```
Code/build_model_dataset.py
```

The script reads two input files: `text_features.csv` and `return_labels.csv`. It uses three merge keys to connect the two tables: `ticker`, `accession_number`, and `filing_date`. These keys identify the same company filing in both datasets. Before merging, the normalization function standardizes the ticker symbols, accession numbers, and filing dates so that small formatting differences do not prevent matching rows from joining correctly.

The function `require_columns()` checks that the necessary merge keys are present in both input files. If one of the required columns is missing, the script stops with a clear error message. This prevents the creation of an incorrect modeling table from incomplete inputs.

By default, the script keeps only rows where the text features have `feature_status = ok` and the return labels have `label_status = ok`. This removes filings where either the document text could not be processed or the stock-price labels could not be calculated. If failed rows should be kept for inspection, the optional command-line argument `--keep-failed` can be used.

The actual merge is performed with an inner join. This means that a filing only appears in the final modeling dataset if it exists in both the text-feature file and the return-label file. The script also uses one-to-one validation, which checks that each filing appears only once in each table. The output is saved as:

```
data/processed/model_dataset.csv
```

This final file contains one row per usable filing, with textual variables on the left side of the row and future return or volatility labels on the right side. It is the dataset used for the later modeling stage. For each company-year filing, the prediction targets include 30-trading-day, 90-trading-day, 180-trading-day, and 252-trading-day forward returns, excess returns relative to SPY, binary outperformance indicators, and realized volatility over the same horizons. The main target in the first version is the 252-trading-day horizon, because it approximately matches a one-year investment period and fits the annual frequency of 10-K filings. Realized volatility is included as a separate target because corporate disclosure language may reveal future uncertainty even when it does not strongly predict the direction of returns.

6 Textual Features and Modeling

6.1 Textual Features

The project should start with interpretable textual features before moving to large language model or transformer-based features.

First, we will only use dictionary-based features. Finance-specific sentiment dictionaries, especially the Loughran-McDonald dictionary, are more appropriate than generic sentiment dictionaries because many words have different meanings in financial contexts. For example, “liability” and “tax” may be negative in common language but are normal financial terms.

Candidate features include negative word share, positive word share, uncertainty word share, litigious word share, modal strong and modal weak word share, report length, average sentence length, readability scores, and the number of risk-related terms.

Next, we consider change-based features, because firms often repeat large parts of their annual reports. Changes from one year to the next may be more informative than the level of sentiment in a single report. Important features include year-over-year change in negative tone, year-over-year change in uncertainty, cosine similarity between the current and previous 10-K, new risk-factor language, and removed or shortened risk disclosures.

Lastly, we also consider embedding-based features. After a strong baseline is built, we add embeddings from financial language models such as FinBERT or general sentence-transformer

models. Since 10-K reports are long documents, they should be split into chunks. Embeddings can then be averaged or pooled by section.

6.2 Modeling Approach

The modeling strategy should begin with simple baselines and gradually add complexity. This makes the project more credible and easier to interpret. The first benchmark is a naive baseline that predicts average historical excess return. The next benchmark uses market information such as previous stock return, volatility, and sector. After that, we compare linear models such as logistic regression or elastic net, tree-based models such as random forest, XGBoost, or LightGBM, and embedding-based models that combine transformer representations with a simple downstream model. This sequence is important because each model answers a slightly different question. A naive baseline asks whether the text model beats a simple historical average. A linear model asks whether the textual features have a stable additive relationship with future returns or volatility. A tree-based model asks whether non-linear thresholds and interactions between features are useful. More complex embedding models would ask whether richer semantic representations of the filing contain information that is missed by dictionary-based features.

Here the split between training and testing is kept quite simple: filings before 2023 are used for training, and filings from 2023 onward are used for testing. This keeps the evaluation closer to the real forecasting problem, where the model must be trained on past filings and evaluated on later filings.

6.3 Baseline Models and First Results

After creating `model_dataset.csv`, the first modeling step is to train simple baseline models. The goal of these models is not to maximize performance immediately, but to establish a transparent benchmark that later feature engineering can be compared against. The script used for this step is:

Code/train_baseline_models.py

The command used in the first experiment is:

```
python Code/train_baseline_models.py \
  --dataset data/processed/model_dataset.csv \
  --output-dir reports/model_results \
  --test-year 2023 \
  --horizon 252
```

The argument `--horizon 252` means that the models use one-year-ahead labels, where one year is approximated by 252 trading days. The argument `--test-year 2023` creates a time-based split: filings before 2023 are used for training, while filings from 2023 onward are used for testing. This is important because it imitates the real research setting, where future filings and future returns are not available when the model is trained.

The code begins by reading `model_dataset.csv` with `pandas`. The function `prepare_data()` identifies the target columns for the selected horizon and checks that the expected textual feature columns are available. The function `time_split()` then creates the training and test sets using the filing year. This split is used separately for each target, and rows with missing target values are dropped before fitting the model. This is necessary because recent filings may not yet have a full 252 trading days of future price data.

For every filing i , the input variables are collected in a feature vector X_i . This vector contains the textual characteristics of the filing: document length, unique word count, type-token ratio, average word length, average sentence length, negative word share, positive word

share, uncertainty word share, litigious word share, and year-over-year changes in selected features. These variables are the explanatory variables of the baseline models. The outcome variables are constructed from future stock prices. The three main outcomes are:

$$Y_i^{class} = Outperform_{i,252},$$

$$Y_i^{ret} = ER_{i,252},$$

$$Y_i^{vol} = \sigma_{i,252}.$$

Here, $Outperform_{i,252}$ is a binary variable equal to one if the stock outperforms SPY over the next 252 trading days, $ER_{i,252}$ is the 252-day excess return relative to SPY, and $\sigma_{i,252}$ is the future realized volatility over the same horizon.

Three prediction tasks are used. The first task is a classification task: predict whether a stock outperforms SPY over the next 252 trading days. The target variable is `outperformed_252d`. For this task, the script trains a logistic regression model and a random forest classifier. Logistic regression is a linear model that estimates how each textual feature changes the probability of future outperformance while holding the other features fixed. The model can be written as:

$$\Pr(Y_i^{class} = 1 \mid X_i) = \frac{1}{1 + \exp[-(\beta_0 + X_i' \beta)]}.$$

The coefficients β describe how the textual variables are associated with the log-odds of future outperformance. If the coefficient on uncertainty share is positive, for example, then higher uncertainty language is associated with a higher predicted probability of outperformance, conditional on the other features. The random forest classifier estimates the same classification target but does not impose a linear equation. Instead, it builds many decision trees that split filings into groups based on feature thresholds and averages their votes. This allows the model to capture interactions, such as uncertainty language mattering only when document length or negative language is also high. In the Python script, this task is implemented in `train_classification()`. The logistic regression is placed inside a `sklearn` pipeline with median imputation and standardization. The random forest classifier is placed inside a pipeline with median imputation only, because tree models do not need standardized variables. The function fits both models on the training set, predicts probabilities on the test set, and saves both the predicted class and the predicted probability.

The second task is a regression task: predict the future 252-day excess return. For this task, the script trains ridge regression and a random forest regressor. Ridge regression is a linear model with regularization, which helps prevent unstable coefficients when features are correlated. The regression equation is:

$$Y_i^{ret} = \alpha + X_i' \beta + \epsilon_i.$$

The model chooses coefficients that minimize squared prediction errors, while the ridge penalty discourages very large coefficients. This is useful because many text features are related to each other: longer documents may also have more risk words, more uncertainty words, and different sentence-length patterns. The random forest regressor estimates the same target, Y_i^{ret} , but through an average of decision trees rather than a single linear equation. It can learn interactions and thresholds, but it is also more prone to fitting noise if the features do not contain a strong signal. In the Python script, this task is implemented with `train_regression()`. For the excess-return target, the function receives `excess_return_252d` as the dependent variable. It fits a ridge regression pipeline with imputation and standardization, and a random forest regression pipeline with imputation. The function then computes mean absolute error, root mean squared error, R^2 , and Spearman rank correlation on the test set. It also compares the models with a simple baseline that always predicts the average excess return observed in the training sample.

The third task is another regression task: predict future 252-day realized volatility. This task tests whether the language in 10-K filings is more informative about future uncertainty than about the direction of returns. The same ridge regression and random forest regressor are used for this target, but the dependent variable changes to:

$$Y_i^{vol} = \sigma_{i,252}.$$

The ridge version again estimates a linear relationship between textual features and future volatility, while the random forest version allows non-linear relationships. Conceptually, this target is important because risk disclosures may be more closely related to the magnitude of future price movement than to whether the price movement is positive or negative. A filing with more uncertainty language could indicate a wider range of possible outcomes, which would appear as higher future realized volatility. The volatility model uses the same `train_regression()` function, but it passes `realized_volatility_252d` as the dependent variable. This means the code structure is identical to the excess-return regression, but the interpretation is different. In this model, a prediction error measures how far the model’s predicted future volatility is from the realized future volatility, rather than how far a predicted return is from the realized return.

The models use the first-pass textual features, including document length, average word length, average sentence length, negative word share, positive word share, uncertainty word share, litigious word share, and year-over-year changes in selected features. Missing feature values are imputed with the median. Linear models also standardize the features before fitting. Standardization matters for linear models because variables such as word count and uncertainty share have very different scales. Tree-based models do not require standardization, but they still use median imputation so that missing year-over-year changes do not remove the first available filing for each company. Finally, the script writes several output files to `reports/model_results`. The main results file contains the metric table shown below. Three prediction files store filing-level predictions for the classification, excess-return, and volatility tasks. A separate correlation file stores simple Pearson and Spearman correlations between each textual feature and each target variable.

Task	Model	Train	Test	Value	Metric
Outperformance	Logistic regression	2226	1397	0.514	ROC-AUC
Outperformance	Random forest	2226	1397	0.491	ROC-AUC
Excess return	Ridge regression	2226	1397	0.289	MAE
Excess return	Random forest	2226	1397	0.292	MAE
Realized volatility	Ridge regression	2226	1397	0.104	MAE
Realized volatility	Random forest	2226	1397	0.101	MAE

Table 1: First baseline model results using 252-trading-day labels and a 2023 time split.

The first results are intentionally interpreted conservatively. For outperformance classification, logistic regression produces a ROC-AUC of 0.514, only slightly above 0.5, while the random forest classifier is below 0.5. This suggests that the current first-pass textual features contain little directional return signal on their own. For excess-return prediction, both models have mean absolute errors close to the simple historical-mean baseline, which indicates that the models do not yet provide strong return forecasts. For realized volatility, the random forest regressor performs slightly better than ridge regression but still does not beat the simple baseline by a meaningful margin.

These results are useful because they establish a first scientific benchmark. At this stage, the evidence does not support a strong predictive relationship between the simple text features and future one-year returns. The next modeling improvements should therefore focus on richer text extraction, finance-specific dictionaries, section-level features such as Item 1A Risk Factors

and Item 7 Management Discussion and Analysis, and stronger controls for firm size, sector, past returns, and historical volatility.

7 Evaluation

The project should evaluate both statistical performance and economic relevance.

7.1 Statistical Metrics

Statistical evaluation should include accuracy for the direction of excess return, ROC-AUC for outperformance classification, mean absolute error for return prediction, and Spearman rank correlation between predicted scores and future returns. Accuracy measures how often the classifier predicts the correct outperformance class, but it can be misleading when one class is much more common than the other. ROC-AUC is therefore also used because it evaluates how well the model ranks outperformers above underperformers across probability thresholds. For return and volatility regression, mean absolute error measures the average absolute size of the prediction mistake in economically interpretable units. Spearman correlation is useful because a model may still be valuable if it ranks firms reasonably well, even when the exact return prediction is noisy.

7.2 Portfolio Backtest

A simple portfolio backtest can make the results easier to understand. Each year, companies are ranked by predicted return score. The analysis can then compare a top-decile portfolio, a bottom-decile portfolio, a long-short portfolio that buys the top decile and sells the bottom decile, and an equal-weighted S&P 500 or SPY benchmark.

The backtest should include transaction-cost sensitivity and should clearly state that it is a research simulation, not investment advice.

8 Repository Structure

The final GitHub repository should be organized as follows:

```
alpha-sentiment-analysis/  
  README.md  
  data/  
    raw/  
    interim/  
    processed/  
  notebooks/  
    01_data_download.ipynb  
    02_text_cleaning.ipynb  
    03_feature_engineering.ipynb  
    04_modeling.ipynb  
    05_portfolio_backtest.ipynb  
  src/  
    data/  
    parsing/  
    features/  
    models/  
    backtest/  
    visualization/
```

```
reports/  
  figures/  
    final_report.md  
app/  
  streamlit_app.py  
requirements.txt  
pyproject.toml
```

9 Ten-Step Project Plan

The project begins by defining the research question, prediction targets, and evaluation rules. It then downloads or imports 10-K filings from SEC EDGAR, cleans the raw filings, and extracts useful text sections. Next, filings are linked to ticker symbols, filing dates, and stock-price data so that forward returns and market-adjusted return labels can be computed. The core modeling dataset is built by creating dictionary-based textual features, then adding change-based and similarity-based features across years. Baseline and machine-learning models are trained using time-based splits, evaluated with statistical metrics, feature importance, and backtests, and finally packaged with a clear README, final report, figures, and an optional Streamlit dashboard.

10 Expected GitHub Deliverables

The final repository should contain a professional README explaining the research question and findings, reproducible notebooks or scripts for each stage of the pipeline, clean feature-engineering code in a reusable Python package structure, model evaluation tables and plots, a portfolio backtest chart, a short final research report, and a Streamlit demo for exploring companies and textual signals.

11 Limitations

Several limitations must be handled honestly. Historical relationships may not persist in the future, public filings are already available to the market when published, and stock returns are noisy and affected by many non-textual factors. Survivorship bias can occur if only current S&P 500 companies are used, while look-ahead bias can occur if filing dates are not handled correctly. Backtests can also overstate performance if transaction costs and liquidity are ignored.

12 Conclusion

Alpha Sentiment Analysis is a portfolio project that combines NLP, finance, machine learning, and reproducible research. Its strongest value is not a promise of easy trading profits, but a transparent scientific test of whether corporate disclosure language contains measurable predictive information about future stock returns.

References

- [1] Tim Loughran and Bill McDonald. *When Is a Liability Not a Liability? Textual Analysis, Dictionaries, and 10-Ks*. Journal of Finance, 66(1):35–65, 2011.
- [2] Narasimhan Jegadeesh and Di Wu. *Word Power: A New Approach for Content Analysis*. Journal of Financial Economics, 110(3):712–729, 2013.

- [3] Lauren Cohen, Christopher Malloy, and Quoc Nguyen. *Lazy Prices*. Journal of Finance, 75(3):1371–1415, 2020.